

Sentiment Analysis Stock Market Prediction System - Technical Implementation Specification

Document Overview

This document provides a comprehensive technical specification for implementing a sentiment analysis stock market prediction system backend. The system will be built using pure Python with no web framework dependencies, following a modular architecture that supports agentic workflows, tool-based operations, and node-based processing pipelines. The implementation will leverage Azure Machine Learning for model training, GPT-Nano for AI coordination, and local LSTM models for predictions.

1. System Architecture

1.1 High-Level Architecture

The system will be structured as a modular Python application with the following architectural layers:

Core Layer: Handles configuration management, database connections, security utilities, and shared resources. This layer provides the foundation upon which all other components operate.

Model Layer: Contains SQLAlchemy definitions for database tables, including users, stock data, sentiment analysis results, predictions, and session management. Each model defines the schema and relationships for persistent data storage.

Service Layer: Implements business logic for authentication, data collection, sentiment analysis, prediction generation, Azure ML integration, and agent coordination. Services encapsulate complex operations and interact with the database through sessions.

Tool Layer: Provides reusable functions and utilities that can be called by agents or nodes. Tools are focused, single-purpose operations like fetching stock data, analyzing sentiment, or generating predictions.

Node Layer: Implements the processing pipeline using a node-based architecture where each node performs a specific task and passes results to subsequent nodes. This enables flexible, modular processing flows.

Agent Layer: Coordinates the overall system behavior using AI-driven decision making. The agent uses GPT-Nano to determine which tools to call, which nodes to execute, and how to process results based on user intent.

1.2 Component Interaction Flow

The system follows a request-response pattern where user inputs trigger agent coordination:

1. User input is received by the main application entry point
2. The agent (GPT-Nano) analyzes the input and determines intent
3. Based on intent, the agent selects appropriate tools and nodes
4. Data flows through the selected nodes, with each node performing transformations
5. Results are aggregated and returned to the user
6. All operations are logged and stored in the database

2. Database Schema Design

2.1 PostgreSQL Database Structure

The system uses PostgreSQL as the primary database, with SQLAlchemy providing ORM capabilities. Alembic manages schema migrations. The following tables will be implemented:

Users Table: Stores user account information including email, username, hashed password, full name, account status, and timestamps. This table serves as the foundation for authentication and authorization.

User Sessions Table: Maintains active user sessions with session tokens, IP addresses, user agents, expiration timestamps, and last activity tracking. This enables secure session management and logout functionality.

Stock Symbols Table: Contains metadata about tracked stock symbols including symbol name, company name, exchange, sector, industry, and active status. This table normalizes stock data across the system.

Stock Data Table: Stores historical stock price data including open, high, low, close, volume, and adjusted close values. Each record is associated with a symbol and date, forming the foundation for technical analysis and predictions.

News Articles Table: Holds news article data including title, content, URL, source, associated stock symbol, and publication timestamp. This table feeds the sentiment analysis pipeline.

Sentiment Analysis Table: Stores sentiment analysis results for each article using both TextBlob and VADER algorithms. Includes polarity scores, subjectivity, compound scores, and overall sentiment classification.

Daily Sentiment Table: Provides aggregated sentiment metrics per symbol per day, including average sentiment scores, article counts, and sentiment distribution. This table enables time-series analysis of market sentiment.

Predictions Table: Records prediction results including predicted prices, confidence scores, trends, and actual values for backtesting. Links predictions to users and stock data for comprehensive tracking.

2.2 Indexing Strategy

To ensure optimal query performance, indexes will be created on frequently queried fields: user emails and usernames, session tokens, stock symbols and dates, article URLs, sentiment symbols and dates, and prediction symbols with dates. Composite indexes will be implemented for queries involving multiple conditions, particularly symbol-date combinations in stock and sentiment data.

2.3 Data Relationships

Foreign key constraints maintain referential integrity across tables. Users have one-to-many relationships with sessions and predictions. Stock symbols have one-to-many

relationships with stock data and predictions. News articles have one-to-one relationships with sentiment analysis results. Daily sentiment aggregates many sentiment analysis records per symbol-date combination.

3. Configuration Management

3.1 Environment Variables

The system uses environment variables for configuration management, loaded from a `.env` file. Configuration categories include:

Application Settings: Application name, version, debug mode, and logging level control overall system behavior.

Database Settings: PostgreSQL connection parameters including server, port, user, password, and database name. Both synchronous and asynchronous connection strings are constructed from these parameters.

Azure OpenAI Settings: Endpoint URL, API key, model name, deployment name, and API version for GPT-Nano integration. These enable AI-powered coordination and natural language understanding.

Azure ML Settings: Endpoint URL and API key for the deployed LSTM model, allowing the system to leverage cloud-based predictions when available.

Security Settings: Secret key for JWT token generation, algorithm specification, and token expiration duration. These secure user authentication and session management.

External API Settings: API keys for news services (NewsAPI) and financial data services (Alpha Vantage) that provide external data sources.

3.2 Configuration Loading Strategy

Configuration is loaded at application startup using Pydantic Settings with validation. The configuration module provides property methods that construct derived values like database URIs from base parameters. This ensures consistent configuration access across all modules while maintaining type safety and validation.

4. Authentication and Security

4.1 Password Management

User passwords are hashed using bcrypt through Passlib. The authentication service provides methods for password hashing and verification, ensuring credentials are never stored in plain text. Password hashing includes salt generation automatically.

4.2 JWT Token System

JSON Web Tokens (JWT) provide stateless authentication for API access. The token payload includes user ID and username with expiration timestamps. Tokens are generated using the HS256 algorithm with a configured secret key. Token verification validates signature, expiration, and payload structure.

4.3 Session Management

In addition to JWT tokens, the system maintains database-backed sessions for user tracking. Session records include IP addresses, user agents, and activity timestamps for security monitoring. Session expiration is enforced both at the token level and database level, with automatic cleanup of expired sessions.

4.4 Authorization Flow

The authentication flow consists of:

1. User provides credentials for login
2. System validates credentials against database
3. Upon success, system creates JWT token and session record
4. Token and session ID returned to user
5. Subsequent requests include token for validation
6. System verifies token and session before processing

5. Data Collection Module

5.1 Stock Data Collection

The stock data collector fetches historical price data from Yahoo Finance and Alpha Vantage APIs. Collection parameters include symbol, date range, and interval (daily, weekly). The collector handles API rate limits, implements retry logic for failures, and stores data with deduplication based on symbol-date combinations.

Collection process:

1. Check database for existing data in requested range
2. Fetch missing data from external APIs
3. Transform API responses to standard format
4. Store new records in stock data table
5. Log collection metrics and errors

5.2 News Article Collection

The news collector retrieves financial news articles from NewsAPI and custom RSS feeds. Collection focuses on articles mentioning specific stock symbols, with configurable lookback periods. Article collection includes title, content, source URL, publication timestamp, and associated symbols.

Collection process:

1. Query news APIs for articles with stock symbol keywords
2. Filter articles by publication date (last 24 hours by default)
3. Check for duplicates using URL as unique identifier
4. Store new articles in news articles table
5. Trigger sentiment analysis for new articles

5.3 Scheduled Collection

The system implements scheduled data collection using the schedule library. Collection jobs run at configurable intervals: stock data daily after market close, news articles every hour, and sentiment analysis immediately after new articles arrive. Scheduled jobs run in background threads, with logging and error handling for reliability.

6. Sentiment Analysis Module

6.1 Dual Algorithm Approach

The sentiment analysis module implements both TextBlob and VADER sentiment analysis algorithms to provide comprehensive sentiment evaluation:

TextBlob Analysis: Provides polarity (-1 to 1) indicating sentiment direction and subjectivity (0 to 1) indicating opinion strength. TextBlob uses a pre-trained sentiment model suitable for general text analysis.

VADER Analysis: Provides compound scores (-1 to 1) and separate positive, neutral, negative scores. VADER is specifically designed for social media and financial text, handling capitalization, punctuation, and emoticons effectively.

6.2 Text Preprocessing

Before analysis, text undergoes preprocessing:

1. Remove HTML tags and special characters
2. Convert to lowercase for consistency
3. Remove extra whitespace
4. Tokenization for analysis
5. Stop word removal (optional based on configuration)

6.3 Combined Sentiment Scoring

The system combines scores from both algorithms:

1. Normalize both scores to -1 to 1 range
2. Calculate weighted average (configurable weights, default 0.5 each)
3. Classify as positive (>0.05), negative (<-0.05), or neutral
4. Store individual and combined scores in database

6.4 Daily Sentiment Aggregation

Daily sentiment aggregation processes all articles for a symbol on a given day:

1. Retrieve all sentiment records for symbol and date
2. Calculate average sentiment score
3. Count articles in each sentiment category
4. Store aggregated results in daily sentiment table
5. Update existing records if new articles arrive for same day

7. Prediction Module

7.1 LSTM Model Architecture

The LSTM model implementation follows a sequential architecture with specific layers:

Input Layer: Accepts sequences of 60 days of historical data, with features including price data, volume, sentiment scores, and technical indicators.

First LSTM Layer: 50 units with return sequences enabled to pass information to subsequent layers. Includes dropout regularization to prevent overfitting.

Second LSTM Layer: 50 units with return sequences disabled, consolidating temporal information into a single vector.

Dense Layers: 25-unit dense layer with ReLU activation, followed by output layer with single unit for price prediction.

Model Compilation: Uses Adam optimizer with 0.001 learning rate and mean squared error loss function.

7.2 Data Preparation for LSTM

Data preparation transforms raw historical data for LSTM input:

Feature Engineering:

- Calculate price returns as percentage changes
- Compute moving averages (5-day, 20-day)
- Calculate rolling volatility (20-day standard deviation of returns)
- Normalize all features to 0-1 range using MinMaxScaler

Sequence Creation:

- Create sequences of 60 consecutive days as input
- Target is the next day's closing price
- Handle missing data through forward filling
- Split into training (80%) and testing (20%) sets

7.3 Local Model Training

When running locally, the system trains LSTM models on demand:

1. Fetch historical stock and sentiment data
2. Engineer features and create sequences
3. Build LSTM model architecture
4. Train for 50 epochs with batch size 32
5. Save model to disk using joblib or TensorFlow SavedModel format
6. Evaluate on test set to calculate baseline accuracy

7.4 Azure ML Integration

The system integrates with Azure ML for cloud-based predictions:

Model Deployment: The LSTM model is trained and deployed as an endpoint in Azure ML, making predictions available via REST API.

Prediction Workflow:

1. Prepare features for the current date
2. Format data as JSON for API request
3. Send POST request to Azure ML endpoint with authentication
4. Parse and validate response
5. Store prediction results locally for caching

Fallback Strategy: If Azure ML endpoint is unavailable or unconfigured, the system falls back to local predictions using the trained LSTM model.

7.5 Prediction Generation

The prediction process generates forecasts for specified time horizons:

Single-Day Prediction:

1. Retrieve last 60 days of data
2. Prepare features and normalize
3. Feed through model (local or Azure)
4. Apply inverse normalization to get actual price
5. Calculate percentage change from current price

Multi-Day Prediction:

1. Generate prediction for day 1
2. Use predicted value as input for day 2 prediction
3. Repeat for specified number of days
4. Apply constraints to prevent unrealistic volatility

Confidence Scoring:

- Calculate based on model's recent prediction accuracy
- Adjust based on sentiment volatility (high volatility reduces confidence)
- Consider market volatility levels
- Output confidence as percentage between 0 and 1

8. Agentic Coordination System

8.1 GPT-Nano Integration

The system uses GPT-Nano as the orchestrator for agent behavior:

Client Setup: AzureOpenAI client configured with endpoint, API key, deployment name, and API version. The client handles authentication and request formatting.

System Prompt Configuration: The agent receives a system prompt defining its role as a financial analyst assistant with access to stock prediction tools. The prompt establishes constraints, capabilities, and response format expectations.

Tool Definitions: Available tools are described to the agent with function names, parameters, and expected outputs. This allows the agent to determine when to call specific functions.

8.2 Intent Recognition

The agent analyzes user input to determine intent:

1. User query is sent to GPT-Nano with context
2. Model returns structured intent classification
3. Intent types include: get prediction, analyze sentiment, view stock data, compare stocks, explain trends, or general questions
4. Extracted entities include stock symbols, timeframes, and specific parameters

8.3 Tool Selection and Execution

Based on recognized intent, the agent selects appropriate tools:

1. Agent chooses from available tools based on intent
2. Tools are called with extracted parameters
3. Results are formatted and returned to agent
4. Agent may make multiple tool calls to gather complete information
5. Final response is synthesized from tool outputs

8.4 Node-Based Processing

The system implements a node-based pipeline architecture:

Node Definition: Each node is a processing unit with inputs, outputs, and transformation logic. Nodes can be connected in sequences to create processing pipelines.

Node Types:

- Data Collection Nodes: Fetch data from external sources
- Preprocessing Nodes: Clean and normalize data
- Analysis Nodes: Perform sentiment analysis or technical calculations
- Prediction Nodes: Generate predictions using ML models
- Aggregation Nodes: Combine results from multiple nodes
- Output Nodes: Format and prepare responses

Pipeline Execution: The agent constructs pipelines dynamically by selecting and connecting nodes based on user intent. Each node executes sequentially, passing outputs to the next node in the pipeline.

9. Tool System

9.1 Stock Data Tools

Fetch Historical Data Tool: Retrieves stock price data for a given symbol and date range. Returns structured data including prices, volumes, and calculated indicators.

Fetch Current Price Tool: Gets the latest available price for a symbol from database or external API if needed.

Get Technical Indicators Tool: Calculates and returns technical indicators including moving averages, RSI, MACD, and Bollinger Bands based on historical data.

9.2 Sentiment Analysis Tools

Analyze Article Sentiment Tool: Performs sentiment analysis on a specific news article and stores results.

Get Daily Sentiment Tool: Retrieves aggregated sentiment for a symbol on a specific date or date range.

Compare Sentiment Tool: Compares sentiment across multiple symbols or time periods, identifying trends and divergences.

9.3 Prediction Tools

Predict Price Tool: Generates price prediction for a symbol with specified time horizon, returning predicted price, confidence, and trend.

Batch Prediction Tool: Generates predictions for multiple symbols or dates in a single operation.

Prediction History Tool: Retrieves historical predictions for a symbol with actual outcomes for backtesting.

9.4 Utility Tools

Search Symbols Tool: Searches for stock symbols matching a query string, returning symbol and company name.

Format Response Tool: Structures tool outputs into consistent JSON format for agent consumption.

Validate Input Tool: Validates tool parameters against expected types and ranges before execution.

10. Database Session Management

10.1 Connection Management

The system maintains both synchronous and asynchronous database connections:

Synchronous Engine: Uses psycopg2 driver with connection pooling for blocking operations. Configured with echo logging for debugging, future mode for SQLAlchemy 2.0 compatibility, and connection pre-ping for health checks.

Asynchronous Engine: Uses asyncpg driver for non-blocking operations. Configured with NullPool to prevent connection pool conflicts in async contexts, and pre-ping for connection validation.

10.2 Session Factories

Session factories provide consistent database session creation:

Sync Session Factory: Creates synchronous sessions bound to the sync engine with auto-commit disabled and auto-flush disabled for transaction control.

Async Session Factory: Creates asynchronous sessions bound to the async engine with the same transaction settings.

10.3 Session Lifecycle

Sessions are managed using context managers and generators:

Sync Session Generator: Yields a session, automatically closing when context exits. Handles commit/rollback based on success/failure.

Async Session Generator: Similar pattern for async operations, ensuring proper cleanup even with exceptions.

Dependency Injection: Session generators are used as dependencies in service classes, providing isolated sessions per operation.

10.4 Migration Management

Alembic handles database migrations:

Migration Workflow:

1. Models are defined in SQLAlchemy
2. Alembic detects changes using environment configuration
3. Migration scripts generated with upgrade/downgrade methods
4. Migrations applied sequentially to database
5. Revision history tracked in database table

Migration Strategy:

- Always create backward-compatible migrations
- Test migrations on development database first
- Backup production database before applying migrations
- Use incremental changes rather than large batch migrations

11. Logging and Monitoring

11.1 Logging Configuration

The system uses loguru for structured logging:

Log Levels: DEBUG for development details, INFO for normal operations, WARNING for recoverable issues, ERROR for failures requiring intervention, CRITICAL for system-wide failures.

Log Format: Timestamp, log level, module name, function name, message, and structured context data.

Log Outputs: Console output for development, rotating file logs for production with daily rotation and compression.

11.2 Operation Logging

Key operations are logged with context:

Data Collection: Log start and end times, symbols collected, articles fetched, errors encountered, and performance metrics.

Sentiment Analysis: Log articles processed, sentiment scores, processing time, and any failures.

Prediction Generation: Log symbols predicted, time horizon, confidence scores, model used (local vs Azure), and execution time.

User Actions: Log authentication attempts, API operations, and session activities for audit purposes.

11.3 Error Tracking

Error handling includes comprehensive logging:

Exception Logging: Full stack traces with context including user ID, operation parameters, and system state at time of error.

Retry Logic: Configurable retry attempts with exponential backoff for transient failures, logged at each attempt.

Alerting: Critical errors trigger console alerts in development, with provisions for external monitoring integration in production.

12. Main Application Entry Point

12.1 Application Initialization

The main application entry point handles system startup:

1. Load configuration from environment
2. Initialize database connection
3. Setup logging
4. Create required directories (models, logs)
5. Initialize agent with GPT-Nano
6. Start background scheduler for data collection
7. Enter main processing loop

12.2 Command-Line Interface

The application provides a CLI using click for user interaction:

Commands:

- run: Start interactive session with agent
- collect-data: One-time data collection for specified symbols
- train-model: Train LSTM model for a symbol
- predict: Generate prediction for a symbol
- analyze-sentiment: Analyze recent articles for sentiment
- backtest: Test prediction accuracy against historical data

Interactive Mode: Provides conversation interface where user queries are processed by the agent, with natural language responses.

12.3 Request Processing Flow

When processing user requests:

1. Parse command line or interactive input
2. Pass to agent for intent recognition
3. Agent selects appropriate tools and nodes
4. Execute tools and nodes sequentially
5. Collect and format results

6. Present response to user
7. Log operation completion

13. LSTM Model Configuration (Awaiting Model)

13.1 Model Integration Strategy

While the sentiment-model.keras file is not yet available, the system is structured to accommodate it when ready:

Model Location: Models will be stored in the app/models/trained/ directory with naming convention {symbol}_lstm.keras for symbol-specific models and a base sentiment-model.keras for the general model.

Model Loading: The prediction service includes a model loader that checks for available model files, with fallback to training on demand if no model exists.

Model Cache: Loaded models are cached in memory to reduce disk I/O for repeated predictions.

13.2 Model Configuration

The configuration for the pending sentiment-model.keras includes:

Input Specification: Expects sequences of 60 days × 9 features (open, high, low, close, volume, sentiment_score, sma_5, sma_20, volatility).

Normalization Parameters: Includes saved MinMaxScaler parameters for consistent feature scaling between training and inference.

Output Format: Provides single-value price prediction, with optional confidence interval outputs if implemented.

Version Tracking: Model version and training metadata stored with model file for tracking and rollback capabilities.

13.3 Model Deployment Workflow

When the model becomes available from Azure:

1. Download model from Azure ML workspace
2. Place in appropriate directory with correct naming
3. System detects model on next prediction request
4. Validate model architecture matches expected structure
5. Load model and use for all predictions
6. Fall back to local training only if model fails to load

13.4 Model Configuration File

A configuration file (model_config.json) will define:

- Available models and their symbols
- Model file paths
- Feature ordering for each model
- Normalization parameters
- Version information
- Training timestamp and metrics

This configuration allows the system to manage multiple models simultaneously and select appropriate models based on symbol and context.

14. Testing Strategy

14.1 Unit Testing

Unit tests will cover individual components:

Service Tests: Test each service method with mock database sessions and external API calls.

Tool Tests: Validate tool functions with known inputs and expected outputs.

Model Tests: Test prediction calculations and data preparation functions with synthetic data.

Utility Tests: Validate helper functions, validators, and formatting logic.

14.2 Integration Testing

Integration tests verify component interactions:

Database Integration: Test actual database operations with test database, including transaction rollbacks.

API Integration: Test external API calls with mock responses, verifying error handling and retry logic.

Agent Integration: Test agent decision-making with predefined queries, verifying tool selection and response quality.

Node Pipeline: Test complete processing pipelines with multiple nodes, ensuring data flows correctly.

14.3 Performance Testing

Performance tests ensure system meets requirements:

Data Collection: Measure time to fetch and store data for various symbols and date ranges.

Sentiment Analysis: Benchmark processing speed for batches of articles.

Prediction Generation: Measure prediction latency with and without model loading.

Concurrent Operations: Test system behavior under multiple simultaneous requests.

15. Deployment Considerations

15.1 Environment Requirements

Python Version: Python 3.12 or higher required for compatibility with async features and type hints.

PostgreSQL: Version 13 or higher for optimal performance with JSON fields and indexes.

Memory Requirements: Minimum 4GB RAM for development, 8GB for production with multiple models loaded.

Storage: 10GB minimum for database and model storage, scaling with data collection volume.

15.2 Configuration for Production

Production deployment requires additional considerations:

Security: Secret keys must use strong random values, stored in environment variables or secret management.

Database: Connection pooling configured with appropriate pool sizes, SSL/TLS for encrypted connections.

Logging: Log rotation with compression, integration with centralized logging systems.

Monitoring: Health check endpoints, performance metrics collection, and alerting setup.

15.3 Backup Strategy

Regular backups protect data and models:

Database: Daily automated backups with point-in-time recovery capability.

Models: Model files backed up separately with version control.

Configuration: Configuration files version-controlled and backed up.

16. Conclusion

This specification provides a comprehensive blueprint for implementing the sentiment analysis stock market prediction system backend. The architecture follows clean separation of concerns with modular components, enabling maintainable and extensible development. The system integrates multiple technologies including PostgreSQL, SQLModel, TensorFlow, Azure ML, and GPT-Nano, all orchestrated through a node-based, agent-driven architecture.

The implementation should proceed in phases, starting with the core database and authentication layers, then building out data collection, sentiment analysis, and prediction capabilities. The agent and node systems can be added incrementally as the foundation stabilizes. The pending LSTM model integration is designed to be seamless, with configuration in place to utilize the model as soon as it becomes available from Azure.

This specification assumes no web framework, using pure Python for backend operations with a command-line interface for user interaction. This approach provides maximum flexibility for the agentic system while maintaining simplicity and performance.